PROJECT-BASED LEARNING

PHASE-II EVALUATION

MEMORA – Saviours of Memory

Workload-Aware Adaptive Buffer Management for Database Systems

Team: Saviours of Memory — Team ID: OS_DBMS_T269

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun

Academic Session 2026–27

Team & Mentor Details

Team Information

Team Name: Saviours of Memory
Project: MEMORA
Team ID: OS_DBMS_T269
Semester: 5th
Section: ML1, ML2, DS1
Domain: DBMS & OS

Team Members

1. Sarthak Tarate – 2026117
2. Shivam Kumar – 2025465
3. Rudra Varun – 2025902

Mentor

Dr. Puneet Sharma

Interactions completed: 3

Phase-I Recap & Feedback Incorporated

Phase-I Status

MEMORA was proposed as a workload-aware buffer manager that watches the page-access stream and changes the eviction policy (LRU, LFU, FIFO) when the workload changes. The four-stage design (buffer pool, phase detector, decision engine, controller) was kept in Phase-II.

Issues Addressed in Phase-II

- Policy switching could flip back and forth on noisy workloads
- Unclear when a switch actually pays off
- Results had no comparison with a real DBMS

Changes Incorporated

- Cost-benefit gate with switching overhead and cold-miss penalty
- Hysteresis, cooldown and minimum net-benefit check
- PostgreSQL validation using `pgbench`

Problem Statement & Finalized Objectives

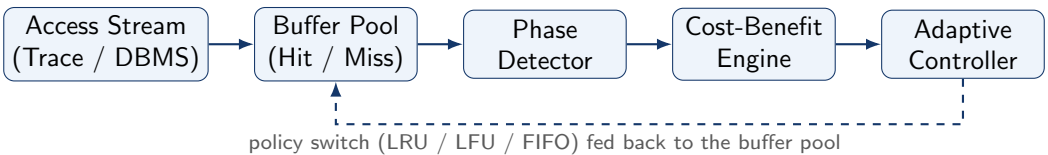
Problem Statement

No single eviction policy is best for every workload: LRU suits recency, LFU suits skewed hot sets and FIFO suits one-pass scans. Real database workloads move between these patterns, so a fixed policy loses hit ratio over time. Switching naively causes thrashing and cold misses. MEMORA switches only when it is worth it.

Finalized Objectives

1. Build a trace-driven buffer pool simulator with LRU, LFU and FIFO eviction
2. Detect workload phase shifts with JS divergence, entropy, sequentiality and reuse distance
3. Switch policy only when expected hit-ratio gain exceeds switching overhead and cold-miss penalty
4. Keep the system stable on noisy workloads using hysteresis, cooldown and net-benefit gating
5. Validate results against PostgreSQL buffer cache statistics

Updated System Architecture / Workflow



Major Components

- Buffer pool: frames, hit/miss, active policy
- Phase detector: sliding-window metrics
- Decision engine: gain vs. cost of switching

Changes Since Phase-I

- Net-benefit gate added before every switch
- Hysteresis and cooldown in the controller
- PostgreSQL cross-validation track added

Module Design & Functional Requirements

Major Modules

1. Buffer Pool Simulation Core
2. Workload Intelligence (Phase Detector)
3. Cost-Benefit Decision Engine
4. Anti-Thrashing Adaptive Controller
5. Evaluation & PostgreSQL Validation

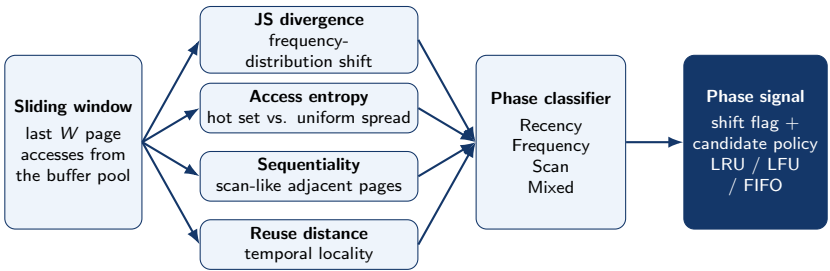
Functional Requirements

- Replay a page-access trace with LRU / LFU / FIFO
- Compute window metrics and flag phase shifts
- Estimate net benefit of switching policy
- Block unstable switches; report hit ratio

Module Dependency

Each access goes through the buffer pool. The phase detector reads the access window, the decision engine scores a possible switch, and the controller decides whether to apply it.

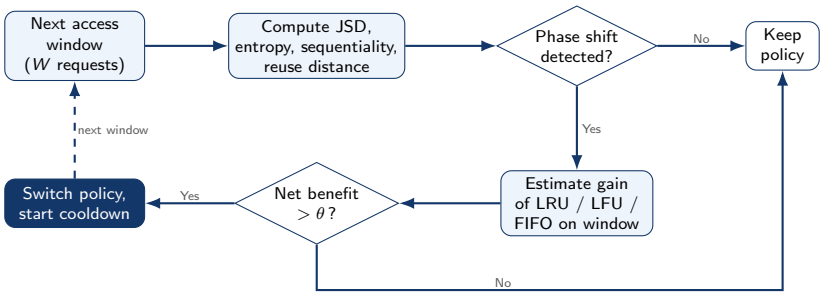
Workload Phase Detection – Block Diagram



How the signals are read

High JS divergence marks a new working set; low entropy means a concentrated hot set (LFU); high sequentiality means a scan (FIFO); short reuse distance favours recency (LRU).

Adaptive Policy-Switching Flowchart



Decision Rule

net benefit = expected hit gain $\times$ window requests $\times$ miss cost $-$ switch overhead $-$ cold-miss penalty. Switch only if net benefit $> \theta$. Hysteresis and cooldown stop back-to-back switches.

Technology Stack & Development Methodology

Technology Stack

- Programming: Python, SQL
- Validation: PostgreSQL, `pgbench`, `pg_stat_statements`
- Testing: pytest
- Development Tool: VS Code
- Version Control: Git / GitHub

Development Methodology

1. Requirement Analysis
2. System Design
3. Module Development
4. Integration
5. Testing & Validation

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures	Hash maps and linked lists (LRU), frequency counters (LFU), queues (FIFO), sliding window	Buffer pool core
Object-Oriented Programming	Common policy interface with LRU, LFU and FIFO as interchangeable classes	Buffer pool core
Operating Systems	Page replacement, locality of reference, thrashing, cold misses	Buffer pool, controller
DBMS	Buffer manager, buffer cache hit/read ratio, workload generation, query statistics	PostgreSQL validation
Probability & Statistics	Jensen–Shannon divergence, entropy, reuse-distance analysis	Phase detector

Development & Implementation Progress

Module	Status	Work Completed / Planned
Buffer Pool Core	Completed	Trace-driven simulation: LRU, LFU, FIFO
Workload Phase Detector	Completed	JSD, entropy, sequentiality, reuse distance
Cost-Benefit Engine	Completed	Gain vs. switching overhead and cold misses
Adaptive Controller	Completed	Hysteresis, cooldown, net-benefit gate
Synthetic Benchmarks	Completed	Recency, Zipfian, scan, mixed, phase-shifting
PostgreSQL Validation	In Progress	`pgbench` runs vs. PostgreSQL statistics
Testing & Tuning	In Progress	pytest suite; threshold tuning
Result Analysis & Plots	Pending	Hit ratio and switch count vs. fixed policies
Final Docs & Demo	Pending	Report, documentation, final demo

Overall Progress: 5 of 9 deliverables completed, 2 in progress, 2 pending

Working Prototype / Demonstration

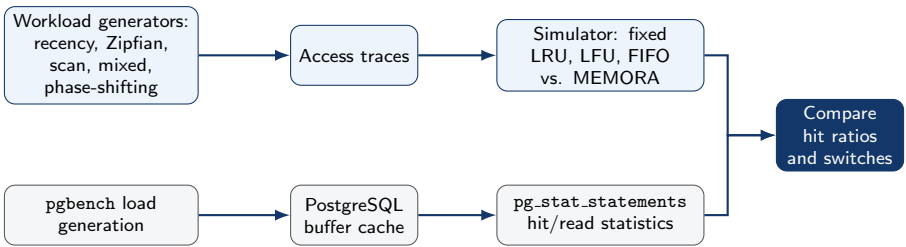
Implemented Features

- Buffer pool simulation with LRU, LFU and FIFO
- Phase-shift detection on a sliding window
- Cost-benefit gated policy switching
- Anti-thrashing control on noisy workloads
- Demo run: `python scripts/run_demo.py`

Prototype / Output

Insert screenshot / output / prototype evidence here

Experimental Setup & Validation Flow



Blue: simulation track. Grey: PostgreSQL track. Both feed the same comparison.

Metrics Compared

Hit ratio, number of policy switches, switching overhead and cold-miss cost, against fixed LRU, fixed LFU and fixed FIFO baselines.

Testing, Results & Validation

Test / Module	Expected Result	Observed Result	Status
Eviction Policies	LRU, LFU, FIFO pick correct victims	Unit tests verified	Pass
Phase Detector	Shift flagged on phase-shifting trace	Shifts found at phase boundaries	Pass
Decision Engine	Switch only if net benefit above threshold	Gating behaviour verified	Pass
Adaptive Controller	No back-to-back switches under noise	Hysteresis and cooldown verified	Pass
PostgreSQL Validation	Simulator trends match PostgreSQL stats	Not yet fully validated	Pending

Results / Validation

4 of 5 test categories passed at module level. Hit-ratio gain over the best fixed policy: *XX%* . PostgreSQL cross-validation and the final comparison follow once the remaining runs are done.

GitHub Repository & Development Activity

Repository Information

- GitHub Link:
  `https://github.com/your-username/memora`
- Branch: main
- README.md: *Updated*
- Contributors: *All Members Added*

Important Commits

- Buffer pool simulation core
- Workload phase detector
- Cost-benefit engine and controller
- Benchmarks and PostgreSQL validation

Individual Contribution

Member	Role	Technical Contribution	Commits
Sarthak Tarate	*Simulation Core*	*Buffer pool, LRU / LFU / FIFO policies, trace replay*	*XX*
Shivam Kumar	*Workload Intelligence*	*Phase detector: JSD, entropy, sequentiality, reuse distance*	*XX*
Rudra Varun	*Decision Engine*	*Cost-benefit engine, controller, PostgreSQL validation*	*XX*

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

- Avoiding thrashing on noisy workloads
- Estimating gain before actually switching
- Tuning thresholds (JSD, cooldown, net benefit)

Solutions / Learning

- Hysteresis, cooldown and net-benefit gating
- Replaying the recent window to score each policy

Phase-III Roadmap

1. Complete PostgreSQL cross-validation with `pgbench`
2. Tune thresholds with parameter sweeps
3. Compare against fixed LRU, LFU and FIFO on all workloads
4. Extra policies (e.g. ARC, CLOCK) if time allows
5. Documentation, final report and demo

Expected Outcomes & References

Expected Final Outcomes

- Working adaptive buffer manager that switches policy by workload phase
- Higher hit ratio than any single fixed policy on phase-shifting workloads
- Fewer unnecessary switches under noise
- Results cross-checked with PostgreSQL statistics

References

1. Silberschatz, Galvin, Gagne – *Operating System Concepts*
2. Elmasri, Navathe – *Fundamentals of Database Systems*
3. PostgreSQL Documentation – `pgbench`
4. Megiddo, Modha – ARC, FAST 2003
5. Lin – Divergence Measures, 1991

Thank You
Questions & Suggestions